Faculty of Engineering
Computers and Systems Engineering Department

EXPERIMENT 8: Building an Employee Management Interface with React

Objective:

In this experiment, you will learn how to use React to consume an API that retrieves employee data, display the data in a grid view, and implement additional functionality like filtering by employee ID and deleting an employee.

Tools Required:

- VS Code
- Node.js (Should be installed)
- A running API that provides employee data

Prerequisite Knowledge:

- Basic knowledge of JavaScript and CSS
- Understanding of React
- Familiarity with RESTful APIs

Experiment Steps:

1. Set Up the React Application

First, create a new folder on your PC. Navigate to that folder, type cmd in the directory path, and press Enter. Then, write down the following commands:

```
npx create-react-app employee-management

cd employee-management
```

2. Create the EmployeeGrid Component

1. **Create a new file** EmployeeGrid.js inside the src directory. This component will handle fetching, displaying, filtering, and deleting employee data.

// src/EmployeeGrid.js

```
import React, { useState, useEffect } from 'react';
import './Employee.css'; // Add a CSS file for styling

function EmployeeGrid() {
  const [employees, setEmployees] = useState([]);
  const [searchId, setSearchId] = useState('');

  // Fetch employee data from the API
  useEffect(() => {
    fetchEmployees();
  }, []);

  const fetchEmployees = () => {
    fetch('your-api-endpoint-url') // Replace with your actual API endpoint
      .then(response => response.json())
      .then(data => {
        const filteredEmployees = data.slice(0,5);

            setEmployees(filteredEmployees)
        // setEmployees(data)
    })
      .catch(error => console.error('Error fetching employee data:', error));
  };
  const fetchEmployeeById = () => {
    if (searchId.trim() === '') {
      fetchEmployees();
    } else {
      fetch(`your-api-endpoint-url/${searchId}`) // Replace with your API endpoint for fetching by ID
        .then(response => response.json())
        .then(data => {setEmployees([data])}) // Assuming the API returns a single object
        .catch(error => console.error('Error fetching employee by ID:', error));
    }
  };
```

```
const deleteEmployee = (id) => {
  fetch(`your-api-endpoint-url/${id}`, { // Replace with your API endpoint for deleting by ID
    method: 'DELETE',
  })
    .then(() => {
      setEmployees(employees.filter(employee => employee.id !== id));
    })
    .catch(error => console.error('Error deleting employee:', error));
};
```

```
  return (
    <div className="employee-grid">
    <input
        type="text"
        placeholder="Enter Employee ID"
        value={searchId}
        onChange={(e) => setSearchId(e.target.value)}
      />
      <button onClick={fetchEmployeeById}>Search</button>
      <table>
        <thead>
          <tr>
            <th> Name</th>
            <th> Email</th>
            <th>Phone</th>
            <th>Salary</th>
          </tr>
        </thead>
        <tbody>
          {employees.map((employee, index) => (
            <tr key={index}>
              <td>{employee.name}</td>
              <td>{employee.email}</td>
              <td>{employee.gender}</td>
              <td>{employee.salary}</td>
              <td>
                <button onClick={() => deleteEmployee(employee.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>);
}

export default EmployeeGrid;
```

3. Style the Component

Create a CSS file Employee.css for basic styling:

```css
.employee-grid {
    width: 80%;
    margin: 20px auto;
    border-collapse: collapse;
  }
  table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
  }

  th, td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #ddd;
  }

  th {
    background-color: #f4f4f4;
  }
  tr:hover {
    background-color: #f1f1f1;
  }
  input[type="text"] {
    padding: 8px;
    margin-right: 8px;
    width: 200px;
  }
  input{
    padding-left: 100px;
  }

  button {
    padding: 8px 12px;
    margin: 10px 0;
  }
```

4. Integrate the EmployeeGrid Component

Now, import and use the EmployeeGrid component in your App.js file:

// src/App.js

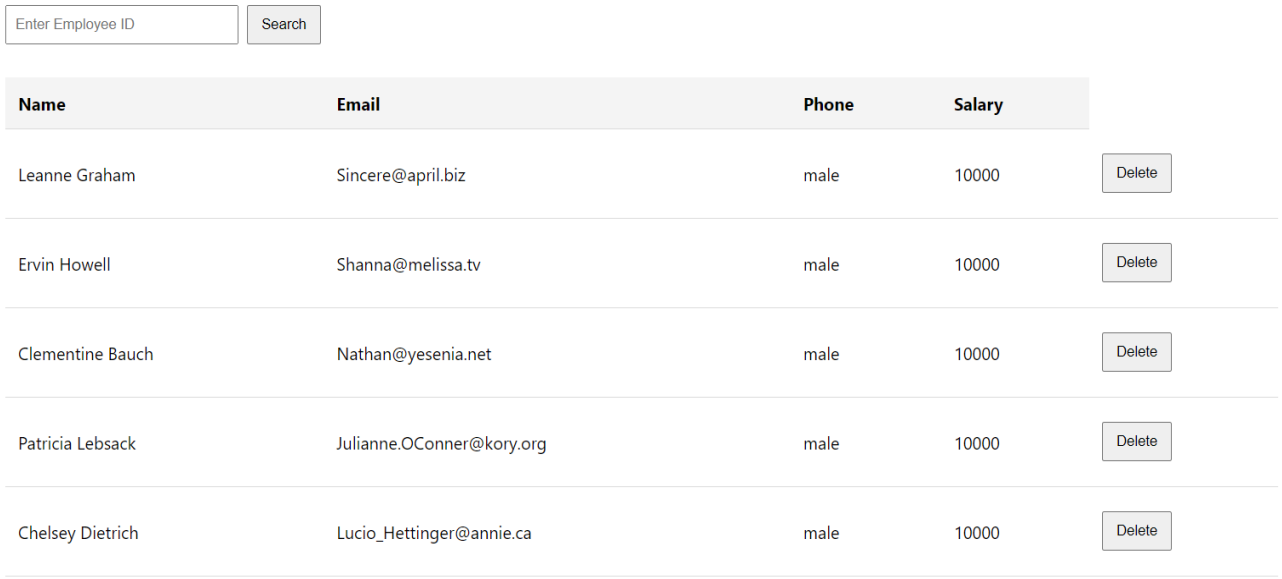
```
import React from 'react';
import EmployeeGrid from './EmployeeGrid';
function App() {
  return (
    <div className="App">
      <h1>Employee Management</h1>
      <EmployeeGrid />
    </div>
  );}
export default App;
```

5. Running the Application

To see the results, run your React application:

```
npm start
```

The output should appear in the browser as follows:



6. Explanation of the Code:

- **State Management (useState)**:
    - employees: Holds the array of employee data fetched from the API.
    - searchId: Holds the value of the employee ID entered by the user.
- **Effect Hook (useEffect)**:
    - The useEffect hook is used to fetch all employee data when the component mounts.
- **Fetching Data**:
    - fetchEmployees: Fetches all employees from the API and sets the employees state.
    - fetchEmployeeById: Fetches a specific employee by ID or fetches all employees if the input is empty.
- **Delete Functionality**:
    - deleteEmployee: Sends a DELETE request to the API to remove an employee and then updates the state to remove the deleted employee from the displayed list.
- **UI Elements**:
    - A text box is provided for entering an employee ID. Upon clicking "Search", the app fetches and displays that specific employee.
    - The grid shows all employees by default and allows deletion with a "Delete" button next to each employee.

7. Student Assignment:

- Modify the EmployeeGrid component to add validation when the employee ID is entered, ensuring that only valid numeric IDs are accepted.
- Add an edit button next to each employee in the grid that allows the user to modify the employee's data (e.g., first name, last name) and save the changes back to the API.
- Implement error handling in the fetchEmployeeById and deleteEmployee functions to show an error message if the API call fails.